

TP n°2

Programmation à objets avancé : langage C++

Opérateurs / Entrées-sorties

L'objectif de ce TP est double. Dans un premier temps il permet d'assimiler correctement le fonctionnement des recopies d'objets lors du cycle de vie d'un objet. L'objectif second est de manipuler le mécanisme de surcharge d'opérateur.

La documentation officielle du langage et de la librairie standard du C++ peut être consultée à cette adresse : <https://en.cppreference.com/>.

1 Cycle de vie d'un objet

1.1 Constructeur de recopie vs opérateur d'affectation par recopie

QUESTION 1. Construire 2 classes *A* et *B* respectant la forme canonique de Coplien et possédant 2 attributs *x* et *y* de type **double** et initialisés par défaut à 0.0.

QUESTION 2. Implémentez toutes les fonctions en effectuant une simple écriture sur la console indiquant la nature de la classe et de la méthode appelée. *L'implémentation du constructeur par défaut est donné en exemple dans le listing 1.*

Listing 1 – Implémentation du constructeur par défaut de la classe A.

```
A::A()  
{  
    cout << "A::A() _ _ Constructeur _ par _ default _ de _ la _ classe _ A" << endl;  
}
```

QUESTION 3. Créez une fonction **main** et recopiez le code fourni dans le listing 2. Commentez les résultats obtenus et observez notamment les fonctions appelées pour les lignes 4 et 5.

Listing 2 – Constructeur, constructeur par recopie et opérateur d'affectation (1).

```
1 A t;  
2 A z(t);  
3 A v=A();  
4 A y=t;  
5 z=t;
```

QUESTION 4. Remplacez maintenant le code de la fonction **main** par celui fourni dans le listing 3. Faites en sorte que le programme puisse compiler et exécutez-le. Commentez les résultats obtenus.

Listing 3 – Constructeur, constructeur par recopie et opérateur d’affection (2).

```
A t;  
B u;  
A a(u);  
t=u;
```

2 Les opérateurs

2.1 Surcharge d’opérateurs

QUESTION 1. Reprendre la classe `Vector` construite au TP précédant et ajouter le constructeur de déplacement `Vector(Vector &&)`.

QUESTION 2. Ajouter les opérateurs d’affectation (de recopie et de mouvement).

QUESTION 3. Ajouter une méthode `void Reserve(unsigned int size)` permettant de modifier la taille d’un vecteur sans en modifier le contenu (redimensionner l’attribut `Vec`).

QUESTION 4. Ajouter les opérateurs d’indexation (`operator[]`) permettant d’accéder à la $i^{\text{ème}}$ coordonnée du vecteur de sorte à pouvoir fonctionner avec le code présenté dans le listing 4.

Listing 4 – Opérateur d’indexation

```
double tab[]={1.0, 2.0, 3.0};  
Vector v(tab,3);  
v[0]=10;  
  
const Vector &cv=v;  
double val=cv[2];
```

2.2 Passage de paramètre et objets temporaires

QUESTION 1. Munissez chacune de vos méthodes d’un affichage permettant de tracer leur exécution. Tester votre classe avec le code fourni dans le listing 5. Repérez les appels aux constructeurs/opérateurs d’affectation/destructeur.

QUESTION 2. Que se passerait-il si nous changions le prototype de la fonction de la ligne 15 du listing 5 par `Vector CreateVector(Vector &p)` ? Par `Vector& CreateVector(Vector p)` ?

QUESTION 3. Ajouter à la classe `Vector` la méthode `Homothecy` qui multiplie chaque dimension du vecteur par une valeur réelle ainsi que la méthode `Norm` qui retourne la norme d’un objet `Vector`. Pour cela, vous pouvez utiliser la fonction `std::sqrt` du langage C (`#include <cmath>`).

QUESTION 4. Ajouter une fonction membre permettant de calculer la distance euclidienne entre 2 vecteurs de même dimension. Faites en sorte que le code du listing 6 puisse être utilisé dans la fonction `main`. Comment accède-t-on aux attributs privés de la classe `Vector` depuis cette fonction ?

Listing 5 – Manipulation des objets temporaires.

```
1 Vector CreateVector(unsigned int nb, ...)
2 {
3     va_list marker;
4     Vector v;
5
6     v.Reserve(nb);
7     va_start(marker, nb);
8     for (unsigned int i = 0; i < nb; ++i)
9         v[i] = va_arg(marker, double);
10    va_end(marker);
11
12    return(v);
13 }
14
15 Vector CreateVector(const Vector &v)
16 {
17     Vector vec(v);
18
19     return(vec);
20 }
21
22 int main()
23 {
24     Vector v1;
25     v1 = CreateVector(4, 1.0, 2.0, 3.0, 4.0);
26
27     Vector v2;
28     v2 = CreateVector(v1);
29 }
```

Listing 6 – Appel de la fonction de calcul de la distance euclidienne de la classe **Vector**.

```
Vector CreateVector(unsigned int nb, ...)
{
    va_list marker;
    Vector v;

    v.Reserve(nb);
    va_start(marker, nb);
    for (unsigned int i = 0; i < nb; ++i)
        v[i] = va_arg(marker, double);
    va_end(marker);

    return(v);
}

int main()
{
    Vector v1,v2;

    v1=CreateVector(2, 1, 2);
    v2=CreateVector(2, 4, 3);

    double dist;
    dist=Vector::EuclideanDistance(v1,v2);
    printf("La_distance_euclidienne_entre_les_vecteurs_v1_et_v2_est_%.f\n", dist)→
        ↵;
}
```